

ບົດທີ 4

ການຄວບຄຸມຄວາມຖືກຕ້ອງຂອງຂໍ້ມູນ

ເນື້ອໃນຫຍໍ້

- ◆ ສະເໜີເບື້ອງຕົ້ນ
- ◆ Integrity Objects
- ◆ Rules
- ◆ Defaults
- ◆ Triggers
- ◆ Constraints

ສະເໜີເບື້ອງຕົ້ນ

- ◆ ການຄວບຄຸມຄວາມຖືກຕ້ອງຂອງຂໍ້ມູນ (Data Integrity) ເປັນການກຳນົດກົດ (Integrity Rule) ເພື່ອຄວບຄຸມຄວາມຖືກຕ້ອງໄວ້ໃນຖານຂໍ້ມູນ ແລະ ໃຊ້ງານກົດນັ້ນໂດຍ DBMS
- ◆ ມີຄວາມສຳຄັນຫຼາຍ ເນື່ອງຈາກມັນຈະເຮັດໃຫ້ໝັ້ນໃຈ ວ່າຂໍ້ມູນທີ່ຖືກເກັບຢູ່ໃນຖານຂໍ້ມູນນັ້ນເປັນຂໍ້ມູນທີ່ ຖືກຕ້ອງ ສາມາດເອົາມາໃຊ້ງານໄດ້ ແລະ ບໍ່ມີຂໍ້ມູນທີ່ ເປັນຂໍ້ມູນເສຍຫາຍປະປົນຢູ່ໃນຖານຂໍ້ມູນ

ສະເໜີເບື້ອງຕົ້ນ

- ◆ ການຄວບຄຸມຄວາມຖືກຕ້ອງ ແບ່ງອອກເປັນ 4 ປະເພດ:
 - Entity Integrity
 - ◆ ເປັນການຄວບຄຸມຄວາມຖືກຕ້ອງໃນລະດັບ Row ຂອງຕາຕະລາງ
 - ◆ ໃຫ້ແຕ່ລະ Row ມີຂໍ້ມູນບໍ່ຊ້ຳກັນ
 - ◆ ສາມາດກຳນົດໂດຍວິທີ ກຳນົດ PK, Unq. Const., Identity
 - Domain Integrity
 - ◆ ເປັນການກຳນົດເງື່ອນໄຂໃນການກວດສອບ ເພື່ອທີ່ຈະກວດສອບຂໍ້ມູນກ່ອນທີ່ຈະຈັດເກັບລົງໃນຖານຂໍ້ມູນ
 - ◆ ຈະຈັດເກັບຂໍ້ມູນທີ່ຖືກຕ້ອງຕາມເງື່ອນໄຂເທົ່ານັ້ນ
 - ◆ ໃຊ້ວິທີການກວດສອບຈາກປະເພດຂອງຂໍ້ມູນແຕ່ລະຖັນ, Ch Constr., Rules, Default obj, NULL-NOTNULL, FK

ສະເໜີເບື້ອງຕົ້ນ

- ◆ ການຄວບຄຸມຄວາມຖືກຕ້ອງ ແບ່ງອອກເປັນ 4 ປະເພດ:
 - Referential Integrity
 - ◆ ເປັນການຄວບຄຸມຄວາມຖືກຕ້ອງໃນສ່ວນຂອງຄວາມສໍາພັນຂອງຕາຕະລາງທີ່ກ່ຽວຂ້ອງກັນລະຫວ່າງ PK ກັບ FK ຫຼື ລະຫວ່າງ FK ກັບ UK ຂອງຕາຕະລາງຫຼັກ ແລະ ຕາຕະລາງສໍາຮອງ
 - User-Define Integrity
 - ◆ ເປັນການກໍານົດການຄວບຄຸມຄວາມຖືກຕ້ອງຂອງຂໍ້ມູນໃນຮູບແບບທີ່ນອກເໜືອຈາກ 3 ປະເພດກ່ອນໜ້ານີ້ ເພື່ອໃຫ້ຂໍ້ມູນທີ່ເກັບນັ້ນເປັນໄປຕາມຄວາມຕ້ອງການຂອງອົງກອນ ໂດຍການກໍານົດ Constraints ໃນລະດັບຄໍລໍາ (ຖັນ)
 - ◆ ເອີ້ນອີກຢ່າງໜຶ່ງວ່າ Business Rules

Integrity Objects

- ◆ ໃນຄໍລໍາຂອງຕາຕະລາງ ນອກຈາກຈະສາມາດກຳນົດປະເພດຂອງຂໍ້ມູນ ແລະ ຂະໜາດຂອງຂໍ້ມູນໄດ້ແລ້ວ ການກຳນົດ Integrity ໃຫ້ກັບຂໍ້ມູນກໍເປັນສິ່ງສໍາຄັນ
- ◆ Object ທີ່ໃຊ້ໃນການຄວບຄຸມຄວາມຖືກຕ້ອງຂອງຂໍ້ມູນປະກອບດ້ວຍ 4 Object ດັ່ງນີ້:
 - Rules
 - Defaults
 - Triggers
 - Constraints

Rules

- ◆ ໃຊ້ໃນການກຳນົດຄ່າຂອງຂໍ້ມູນທີ່ຮັບມາວ່າກົງກັບເງື່ອນໄຂທີ່ຕັ້ງໄວ້ຫຼືບໍ່
- ◆ ໃຊ້ໃນການກວດສອບຂໍ້ມູນທີ່ລະດັບຄໍລໍາຂອງແຕ່ລະ Row
- ◆ ຮູບແບບຄຳສັ່ງ

```
CREATE RULE [schema_name, ] rule_name  
AS condition_expression
```

Rules

◆ ຕົວຢ່າງ ການສ້າງ Rules ແລະ ການກຳນົດ Rules ໃຊ້ງານ

```
CREATE RULE id_chk AS @id BETWEEN 0 and 10000;
GO
CREATE TABLE cust_sample
(
    cust_id                int PRIMARY KEY,
    cust_name              char(50),
    cust_address           char(50),
    cust_credit_limit      money,
);
GO
EXEC sp_bindrule id_chk, 'cust_sample.cust_id'
```


Rules

- ◆ ໃນການໃຊ້ງານ Rule ນັ້ນ ບໍ່ຈໍາເປັນຕ້ອງສ້າງກ່ອນ ຕາຕະລາງກໍໄດ້ ເຮົາສາມາດໃຊ້ງານ Rule ກັບ ຕາຕະລາງທີ່ມີຂໍ້ມູນຢູ່ແລ້ວ
- ◆ ແຕ່ມີຂໍ້ຈຳກັດຢູ່ວ່າ ມັນຈະມີຜົນບັງຄັບໃຊ້ກັບຂໍ້ມູນທີ່ ເຂົ້າມາໃໝ່ຂອງຕາຕະລາງເທົ່ານັ້ນ ສ່ວນຂໍ້ມູນເດີມ ກ່ອນການກຳນົດ Rule ນັ້ນຈະບໍ່ມີຜົນ

Defaults

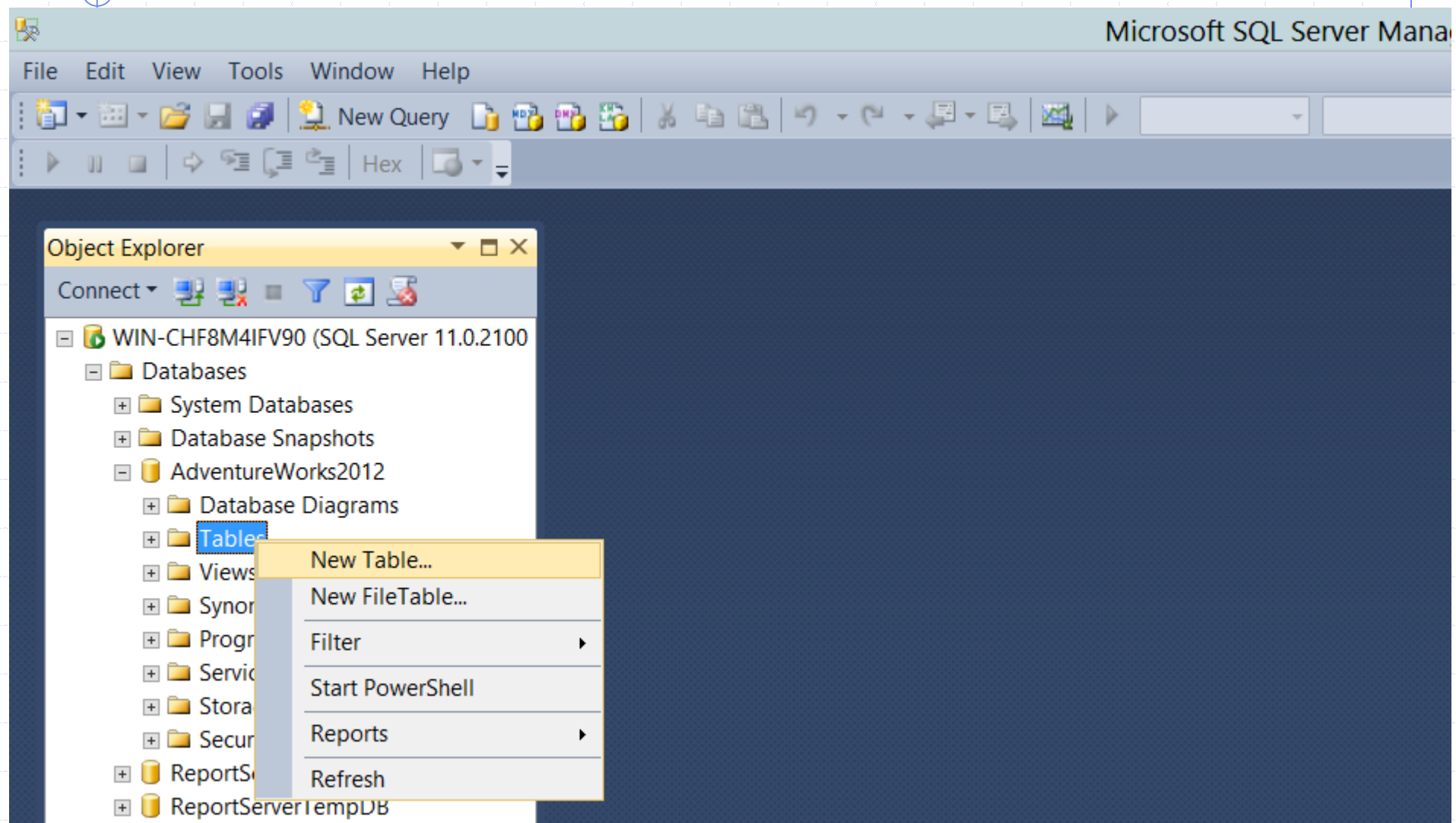
- ◆ ໃຊ້ໃນການກຳນົດຄ່າໃຫ້ກັບຄໍລ່າທີ່ບໍ່ໄດ້ກຳນົດຄ່າໃນຕອນທີ່ມີການ INSERT Row ໃໝ່ໃນຕາຕະລາງ
- ◆ ໃນການກຳນົດ Defaults ໃຫ້ກັບຄໍລ່ານັ້ນ ສາມາດເຮັດໄດ້ 3 ກໍລະນີ
 - ຄ່າຄົງທີ່ (Constant) ເຊັ່ນ 'ABC', 0 ເປັນຕົ້ນ
 - Build-in Function ເຊັ່ນ getdate() ເປັນຕົ້ນ
 - ສູດຄຳນວນທາງຄະນິດສາດ ເຊັ່ນ $10*2$ ເປັນຕົ້ນ
- ◆ ສາມາດເຮັດໄດ້ຕອນສ້າງຕາຕະລາງໃໝ່ ຫຼື ເຂົ້າໄປກຳນົດໃນຕາຕະລາງທີ່ມີຢູ່ແລ້ວກໍໄດ້

Defaults

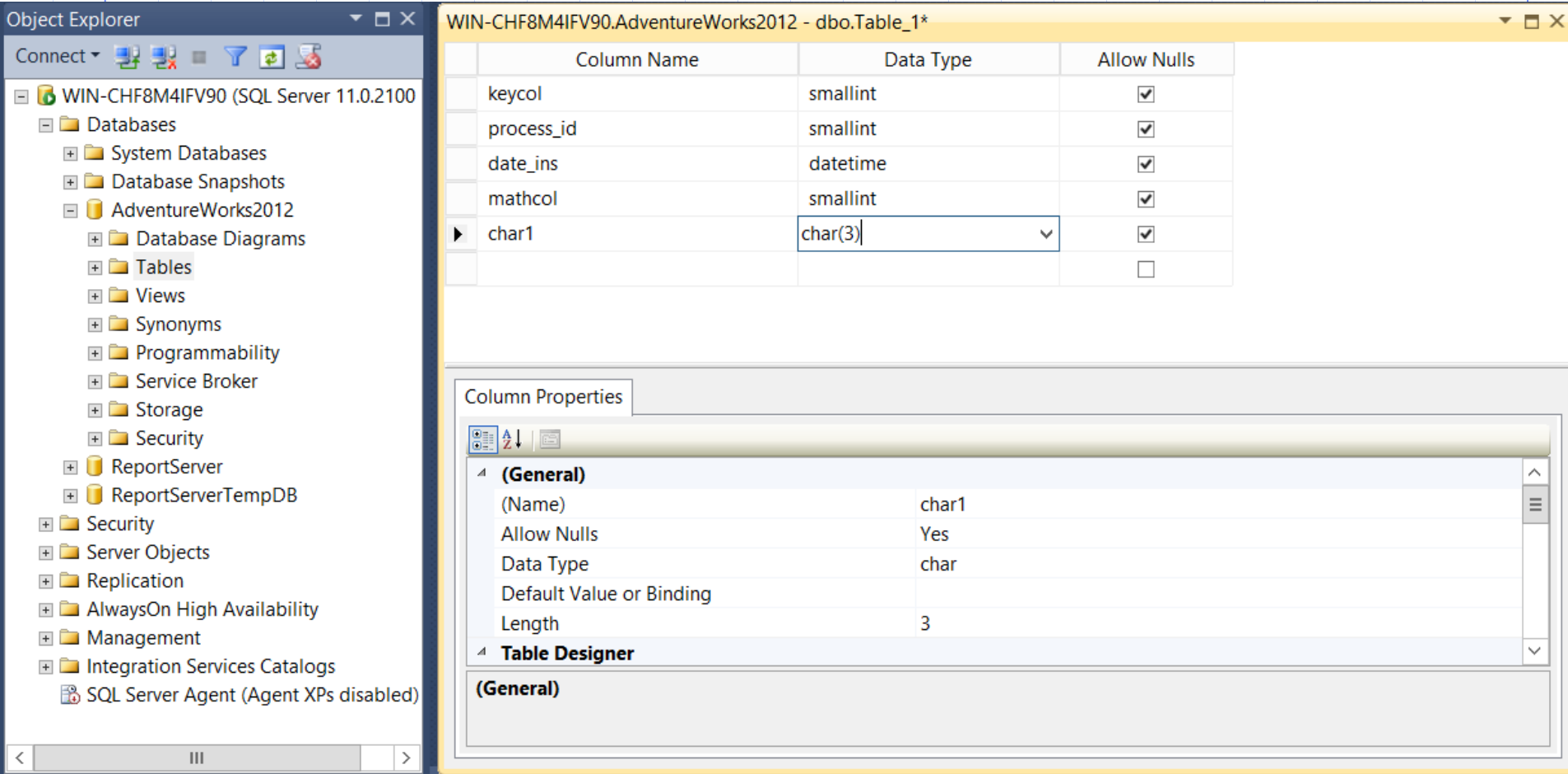
◆ ຂັ້ນຕອນໃນການກຳນົດ Defaults ໃນຕອນສ້າງຕາຕະລາງໃໝ່ມີດັ່ງນີ້

1. ເປີດ SQL Server Management Studio
2. ເຂົ້າໄປໃນຖານຂໍ້ມູນທີ່ຕ້ອງການ ຈາກນັ້ນໃຫ້ຄິກຂວາທີ່ໂຟນເດີ Table ເລືອກ New Table... ເພື່ອສ້າງຕາຕະລາງໃໝ່
3. ໃນໜ້າຈໍການສ້າງຕາຕະລາງໃໝ່ ໃຫ້ເລືອກຄໍລ່າທີ່ຕ້ອງການ ແລ້ວກຳນົດຄ່າ Default ໃນຊ່ອງ Default value

Defaults



Defaults



Object Explorer

Connect

WIN-CHF8M4IFV90 (SQL Server 11.0.2100)

- Databases
 - System Databases
 - Database Snapshots
 - AdventureWorks2012
 - Database Diagrams
 - Tables
 - Views
 - Synonyms
 - Programmability
 - Service Broker
 - Storage
 - Security
 - ReportServer
 - ReportServerTempDB
 - Security
 - Server Objects
 - Replication
 - AlwaysOn High Availability
 - Management
 - Integration Services Catalogs
 - SQL Server Agent (Agent XPs disabled)

WIN-CHF8M4IFV90.AdventureWorks2012 - dbo.Table_1*

Column Name	Data Type	Allow Nulls
keycol	smallint	☒
process_id	smallint	☒
date_ins	datetime	☒
mathcol	smallint	☒
char1	char(3)	☒

Column Properties

(General)

(Name)	char1
Allow Nulls	Yes
Data Type	char
Default Value or Binding	
Length	3

Table Designer

(General)

Defaults

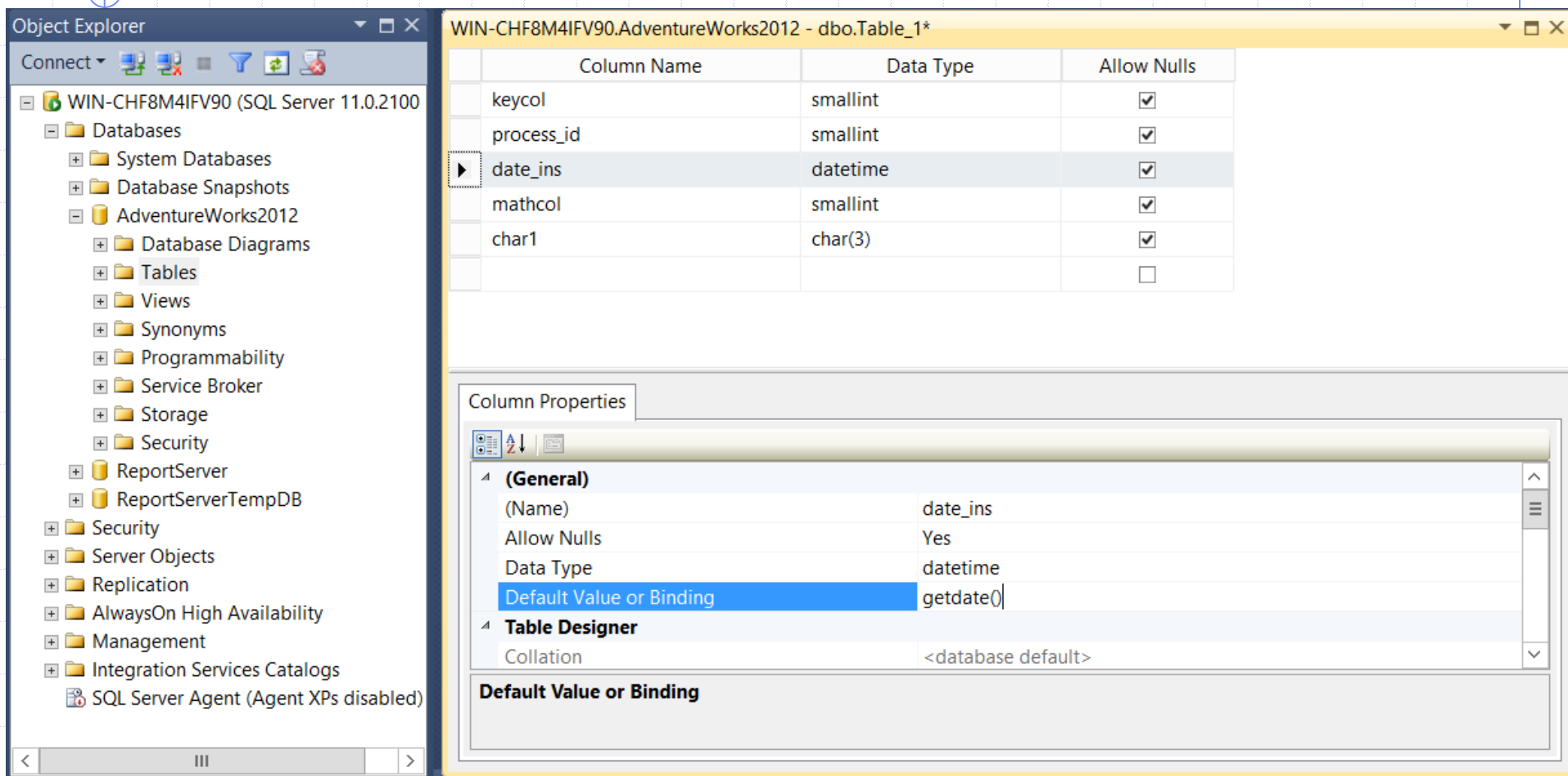
The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane shows the 'AdventureWorks2012' database selected. The main pane shows the 'Table Designer' for 'process_id' in the 'dbo' schema. The table structure is as follows:

Column Name	Data Type	Allow Nulls
keycol	smallint	☒
process_id	smallint	☒
date_ins	datetime	☒
mathcol	smallint	☒
char1	char(3)	☒
		☐

The 'Column Properties' pane for 'process_id' is shown below:

Column Properties	
(General)	
(Name)	process_id
Allow Nulls	Yes
Data Type	smallint
Default Value or Binding	@@spid
Table Designer	
Collation	<database default>

Defaults



The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane shows the 'AdventureWorks2012' database selected under 'Databases'. The 'Tables' folder is expanded, and the 'date_ins' table is selected. The main pane shows the 'Table Designer' for 'date_ins'. The 'Column Properties' tab is active, and the 'Default Value or Binding' sub-tab is selected. The 'Default Value or Binding' property is set to 'getdate()'. The 'Table Designer' also shows the 'Collation' property set to '<database default>'.

Column Name	Data Type	Allow Nulls
keycol	smallint	☒
process_id	smallint	☒
date_ins	datetime	☒
mathcol	smallint	☒
char1	char(3)	☒

Column Properties

(General)

(Name) date_ins

Allow Nulls Yes

Data Type datetime

Default Value or Binding getdate()

Table Designer

Collation <database default>

Default Value or Binding

Defaults

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' shows the 'AdventureWorks2012' database structure, with 'Tables' expanded. The main window shows the 'Table Designer' for 'Table_1' in the 'AdventureWorks2012' database. The 'Column Properties' tab is active, showing the 'Default Value or Binding' for the 'mathcol' column as '10*20'.

Column Name	Data Type	Allow Nulls
keycol	smallint	☒
process_id	smallint	☒
date_ins	datetime	☒
mathcol	smallint	☒
char1	char(3)	☒
		☐

Column Properties

(General)

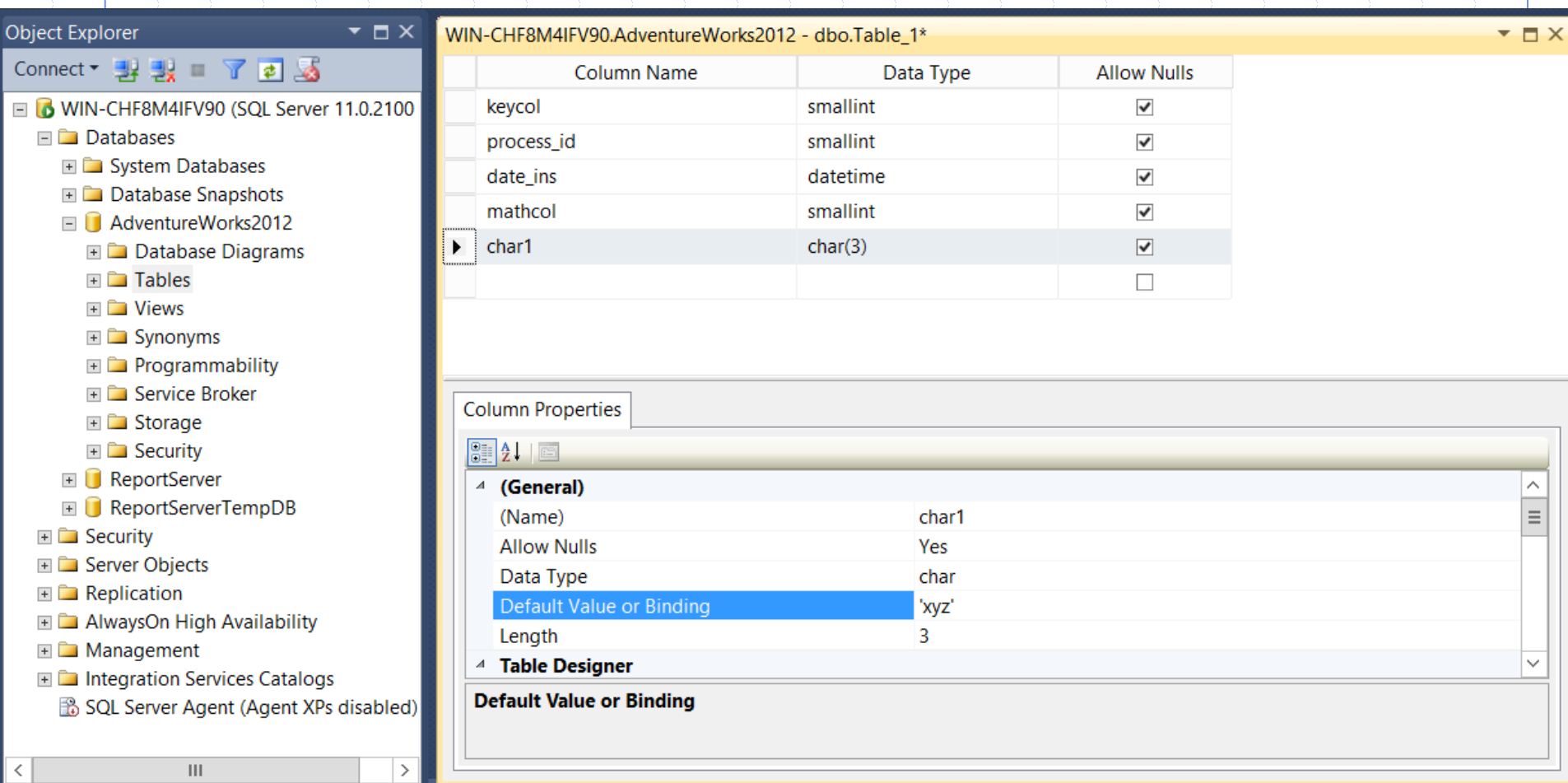
(Name)	mathcol
Allow Nulls	Yes
Data Type	smallint
Default Value or Binding	10*20

Table Designer

Collation	<database default>
-----------	--------------------

Default Value or Binding

Defaults



The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane shows the server 'WIN-CHF8M4IFV90 (SQL Server 11.0.2100)' with the 'AdventureWorks2012' database selected. The 'Tables' folder is expanded, and 'Table_1' is highlighted. The main pane shows the 'Table Properties' window for 'Table_1'. The 'Column Properties' tab is active, and the 'Default Value or Binding' property for the 'char1' column is highlighted. The property is set to 'xyz' with a length of 3.

Column Name	Data Type	Allow Nulls
keycol	smallint	☒
process_id	smallint	☒
date_ins	datetime	☒
mathcol	smallint	☒
char1	char(3)	☒

Column Properties

(General)

(Name)	char1
Allow Nulls	Yes
Data Type	char
Default Value or Binding	'xyz'
Length	3

Table Designer

Default Value or Binding

WIN-CHF8M4IFV90.AdventureWorks2012 - dbo.Table_1*

Column Name	Data Type	Allow Nulls
keycol	smallint	☒
process_id	smallint	☒
date_ins	datetime	☒
mathcol	smallint	☒
char1	char(3)	☒
		☐

Column Properties

(General)

(Name)	char1
Allow Nulls	Yes
Data Type	char
Default Value or Binding	'xyz'
Length	3

Table Designer

Default Value or Binding

Choose Name

Enter a name for the table:

test_defaults

OK Cancel

Defaults

- ◆ ສໍາຫຼັບການໃຊ້ຄໍາສັ່ງ Transact-SQL ກໍານົດຄ່າ Default ໃຫ້ປະຕິບັດດັ່ງນີ້
 - ໃນຄໍລໍາທີ່ຕ້ອງການກໍານົດຄ່າ Default ຢູ່ຫຼັງປະເພດຂໍ້ມູນ ໃຫ້ໃຊ້ຄໍາສັ່ງ DEFAULT ແລ້ວຕາມດ້ວຍຄ່າຂອງ Default ທີ່ຕ້ອງການ

Defaults

◆ ຕົວຢ່າງການໃຊ້ຄໍາສັ່ງ Transact-SQL

```
USE Adventureworks2012
```

```
GO
```

```
CREATE TABLE test_defaults
```

```
(
```

```
    Keycol smallint,
```

```
    Process_id smallint DEFAULT @@SPID,
```

```
    Date_ins datetime DEFAULT getdate(),
```

```
    Mathcol smallint DEFAULT 10*20,
```

```
    Char1 char(3) DEFAULT 'xyz'
```

```
)
```

```
GO
```

```
INSERT INTO test_defaults(keycol) VALUES (1)
```

Defaults

◆ ຕົວຢ່າງການໃຊ້ຄໍາສັ່ງ Transact-SQL

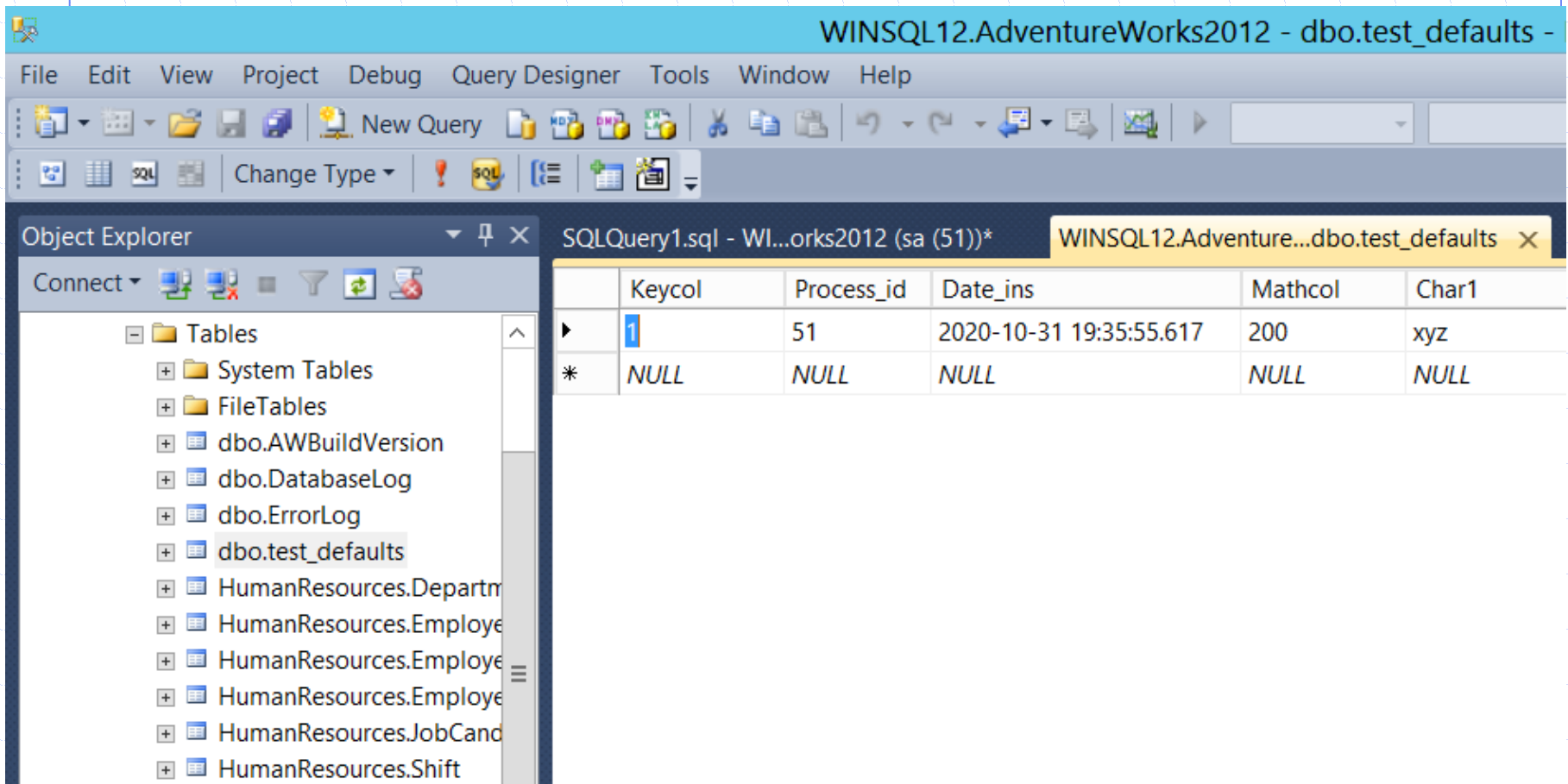
The screenshot displays the Microsoft SQL Server Enterprise Manager interface. The title bar indicates the connection to 'WINSQL12.AdventureWorks2012 (sa (51))'. The menu bar includes File, Edit, View, Query, Project, Debug, Tools, Window, and Help. The toolbar contains icons for New Query, Open, Save, Print, Undo, Redo, and Execute. The Object Explorer on the left shows the database structure, with 'dbo.test_defaults' selected under the 'Tables' folder. The SQL Query window on the right contains the following Transact-SQL script:

```
USE Adventureworks2012
GO
CREATE TABLE test_defaults
(
    Keycol smallint,
    Process_id smallint DEFAULT @@SPID,
    Date_ins datetime DEFAULT getdate(),
    Mathcol smallint DEFAULT 10*20,
    Char1 char(3) DEFAULT 'xyz'
)
GO
INSERT INTO test_defaults(keycol) VALUES (1)
```

The Messages pane at the bottom shows the result: '(1 row(s) affected)'.

Defaults

◆ ຕົວຢ່າງການໃຊ້ຄໍາສັ່ງ Transact-SQL



The screenshot displays the Microsoft SQL Server Enterprise Manager interface. The title bar indicates the connection is to 'WINSQL12.AdventureWorks2012 - dbo.test_defaults'. The menu bar includes File, Edit, View, Project, Debug, Query Designer, Tools, Window, and Help. The toolbar contains icons for New Query, Open, Save, Print, and other standard database operations. The Object Explorer on the left shows the database structure, with 'dbo.test_defaults' selected under the 'Tables' folder. The main window shows the results of a query in 'SQLQuery1.sql - WI...orks2012 (sa (51))*'. The query results are displayed in a table with the following data:

	Keycol	Process_id	Date_ins	Mathcol	Char1
▶	1	51	2020-10-31 19:35:55.617	200	xyz
*	NULL	NULL	NULL	NULL	NULL

Triggers

- ◆ ເປັນ store procedure ຊະນິດພິເສດທີ່ເຮັດວຽກເອງແບບອັດຕະໂນມັດ ເມື່ອມີເຫດການທີ່ກົງກັບເງື່ອນໄຂຂອງມັນ
- ◆ ໃນ SQL Server 2012 ມີ 3 Triggers ຊະນິດ
 - DDL Trigger
 - DML Trigger
 - CLR Trigger

Triggers

◆ DDL Trigger

- ເປັນ Triggers ທີ່ເຮັດວຽກເມື່ອມີຄໍາສັ່ງໃນກຸ່ມ DDL ເຮັດວຽກເຊັ່ນ CREATE TABLE
- ການສ້າງ DDL Trigger ມີຮູບແບບດັ່ງນີ້

```
CREATE TRIGGER trigger_name
ON {ALL SERVER | DATABASE}
[WITH <ddl_trigger_option> [...n]]
{FOR|AFTER} {event_type|event_group} [...n]
AS {sql_statement[:][,...n]|EXTERNAL NAME <method specifier>[:]}
<ddl_trigger_option>::=
    [ENCRYPTION]
    [EXECUTE AS Clause]
<method_option>::=
    assembly_name.class_name.method_name
```


Triggers

◆ DDL Trigger

- ສ່ວນຫຼາຍມັກໃຊ້ໃນການປ້ອງກັນການລຶບ, ການແກ້ໄຂຕາຕະລາງ
- ຕົວຢ່າງການສ້າງ Trigger ປ້ອງກັນການລຶບ, ການແກ້ໄຂຕາຕະລາງ

```
CREATE TRIGGER safety
```

```
ON DATABASE
```

```
FOR DROP_TABLE, ALTER_TABLE
```

```
AS
```

```
PRINT 'You must disable Trigger "safety" to drop or alter table'
```

```
ROLLBACK
```

Triggers

◆ DDL Trigger

- ສາມາດກຳນົດໃຫ້ບັນທຶກການເຮັດວຽກດ້ວຍຄຳສັ່ງ DDL ກັບຖານຂໍ້ມູນໄດ້
- ເຮັດໃຫ້ DBA ສາມາດກວດສອບໄດ້ວ່າ ມີຜູ້ໃຊ້ຄົນໃດໃຊ້ງານຄຳສັ່ງ DDL ກັບຖານຂໍ້ມູນ
- ການກຳນົດ DDL Trigger ບັນທຶກການເຮັດວຽກຂອງຄຳສັ່ງກຸ່ມ DDL ໃນຖານຂໍ້ມູນ ມີ 2 ຂັ້ນຕອນດັ່ງນີ້
 1. ສ້າງຕາຕະລາງບັນທຶກໃນຖານຂໍ້ມູນທີ່ຕ້ອງການ
 2. ສ້າງ Trigger ໃນຖານຂໍ້ມູນທີ່ຕ້ອງການ
- ຫຼັງຈາກສ້າງ Trigger ແລ້ວໃຫ້ທົດລອງເພີ່ມ ຫຼື ແກ້ໄຂ ແລ້ວລຶບຕາຕະລາງໃນຖານຂໍ້ມູນ ແລ້ວເປີດຂໍ້ມູນໃນຕາຕະລາງ DatabaseLog

Triggers

◆ DDL Trigger

■ ສ້າງຕາຕະລາງບັນທຶກໃນຖານຂໍ້ມູນທີ່ຕ້ອງການ

USE test_filegroup;

```
CREATE TABLE dbo.DatabaseLog(  
    DatabaseLogID int IDENTITY(1,1) NOT NULL,  
    PostTime datetime NOT NULL,  
    DatabaseUser sysname NOT NULL,  
    [Event] sysname NOT NULL,  
    [Schema] sysname NOT NULL,  
    Object sysname NULL,  
    TSQL nvarchar(max) NOT NULL,  
    XmlEvent xml NOT NULL,  
    CONSTRAINT PK_DatabaseLog_DatabaseLogID PRIMARY KEY NONCLUSTERED  
    (  
        DatabaseLogID ASC  
    ) WITH (IGNORE_DUP_KEY=OFF) ON [PRIMARY]  
    ) ON [PRIMARY]
```

Triggers

◆ DDL Trigger

■ ສ້າງ Trigger ໃນຖານຂໍ້ມູນທີ່ຕ້ອງການ

```
USE test_filegroup  
GO
```

```
CREATE TRIGGER [ddlDatabaseTriggerLog] ON DATABASE  
FOR DDL_DATABASE_LEVEL_EVENTS AS  
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    DECLARE @data XML;  
    DECLARE @schema sysname;  
    DECLARE @object sysname;  
    DECLARE @eventType sysname;
```

```
    SET @data = EVENTDATA();  
    SET @eventType = @data.value('/EVENT_INSTANCE/EventType)[1]', 'sysname');  
    SET @schema = @data.value('/EVENT_INSTANCE/SchemaName)[1]', 'sysname');  
    SET @object = @data.value('/EVENT_INSTANCE/ObjectName)[1]', 'sysname');
```

Triggers

◆ DDL Trigger

■ ສ້າງ Trigger ໃນຖານຂໍ້ມູນທີ່ຕ້ອງການ

```
IF @object IS NOT NULL
    PRINT ' ' + @eventType + ' - ' + @schema + '.' + @object;
ELSE
    PRINT ' ' + @eventType + ' - ' + @schema;

IF @eventType IS NULL
    PRINT CONVERT(nvarchar(max), @data);

INSERT [dbo].[DatabaseLog]
(
    [PostTime],
    [DatabaseUser],
    [Event],
    [Schema],
    [Object],
    [TSQL],
    [XmlEvent]
)
```

Triggers

◆ DDL Trigger

■ ສ້າງ Trigger ໃນຖານຂໍ້ມູນທີ່ຕ້ອງການ

```
VALUES
(
  GETDATE(),
  CONVERT(sysname, CURRENT_USER),
  @eventType,
  CONVERT(sysname, @schema),
  CONVERT(sysname, @object),
  @data.value('/EVENT_INSTANCE/TSQLCommand)[1]', 'nvarchar(max)'),
  @data
);
END;
```

Triggers

◆ DDL Trigger

- ຫຼັງຈາກທີ່ໄດ້ສ້າງ Trigger ແລ້ວລອງສ້າງຕາຕະລາງ, ແກ້ໄຂ ແລະ ລຶບຕາຕະລາງ ຫຼັງຈາກນັ້ນໃຫ້ເປີດຕາຕະລາງ DatabaseLog ຈະມີຂໍ້ມູນດັ່ງຮູບ

WINSQL12.test_file...- dbo.DatabaseLog x SQLQuery1.sql - WIN...filegroup (sa (51))*

	Databas...	PostTime	Database...	Event	Schema	Object	TSQL	XmlEvent
▶	1	2020-10-31 21:03:00.040	dbo	CREATE_TABLE	dbo	Students	CREATE TAB...	<EVENT_IN...
	2	2020-10-31 21:03:00.060	dbo	ALTER_TABLE	dbo	Students	ALTER TABL...	<EVENT_IN...
	3	2020-10-31 21:05:29.423	dbo	ALTER_TABLE	dbo	Students	ALTER TABL...	<EVENT_IN...
	4	2020-10-31 21:05:29.423	dbo	ALTER_TABLE	dbo	Students	ALTER TABL...	<EVENT_IN...
	5	2020-10-31 21:05:46.583	dbo	DROP_TABLE	dbo	Students	DROP TABLE...	<EVENT_IN...
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Triggers

◆ DDL Trigger

■ ການແກ້ໄຂ DDL Trigger

- ◆ ການແກ້ໄຂ Trigger ນັ້ນ ຈະຕ້ອງໃຊ້ຄໍາສັ່ງ Transact-SQL ມີຮູບແບບດັ່ງນີ້

```
ALTER TRIGGER trigger_name
ON {DATABASE|ALL SERVER}
[WITH <ddl_trigger_option> [...n]]
{FOR | AFTER} {event_type [...n] | event_group}
AS {sql_statement [;] | EXTERNAL NAME <method specifier>[;]}

<ddl_trigger_option>::=
    [ENCRYPTION]
    [EXECUTE AS Clause]

<method_option>::=
    assembly_name.class_name.method_name
```


Triggers

◆ DDL Trigger

■ ການແກ້ໄຂ DDL Trigger

- ◆ ຕົວຢ່າງ ການແກ້ໄຂ DDL Trigger ດ້ວຍຄໍາສັ່ງ Transact-SQL

```
ALTER TRIGGER safety  
ON DATABASE  
FOR DROP_TABLE  
AS
```

```
PRINT 'You must disable Trigger “safety” to drop tables’  
ROLLBACK
```

Triggers

◆ DDL Trigger

■ ການລຶບ DDL Trigger

- ◆ ໃນຖານຂໍ້ມູນ ໃຫ້ເຂົ້າໄປໃນໂຟນເດີ Programmability ແລ້ວເຂົ້າໄປໃນໂຟນເດີ Database Triggers
- ◆ ໃຫ້ຄົກຂວາໃສ່ຊື່ Trigger ທີ່ຕ້ອງການ ແລ້ວເລືອກຄໍາສັ່ງ Delete
- ◆ ຄົກປຸ່ມ OK ເພື່ອຢືນຢັນການລຶບ

■ ການລຶບ DDL Trigger ໂດຍໃຊ້ຄໍາສັ່ງ Transact-SQL ດັ່ງນີ້

```
DROP TRIGGER trigger_name [...n]
```

```
ON {DATABASE | ALL SERVER} [;]
```

- ◆ ຕົວຢ່າງ

```
USE test_filegroup
```

```
GO
```

```
DROP TRIGGER safety ON DATABASE ON DATABASE;
```

- Programmability
 - + Stored Procedures
 - + Functions
 - Database Triggers
 - ddlDatabaseTriggerLog
 - safety

- + Assemblies
- + Types
- + Rules
- + Defaults
- + Plan Guides
- + Sequences
- + Service Broker
- + Storage
- + Security
- + test_filegroup
- + Security
- + Server Objects

- New Database Trigger...
- Script Database Trigger as ▶
- View Dependencies
- Enable
- Disable
- Policies ▶
- Facets
- Start PowerShell
- Reports ▶
- Delete
- Refresh

100 %

Message
Command

100 %

Query editor

Triggers

◆ DML Trigger

- ເປັນ Trigger ທີ່ຈະເຮັດວຽກເມື່ອມີການໃຊ້ງານຄໍາສັ່ງກຸ່ມ DML (INSERT, UPDATE, DELETE) ໃນ ຕາຕະລາງ ຫຼື ວິວ
- ມັກຖືກໃຊ້ງານຄວບຄຸມຄວາມຖືກຕ້ອງຂອງຂໍ້ມູນ ແລະ ໃຊ້ໃນການເຮັດໃຫ້ຂໍ້ມູນເປັນໄປຕາມ Business Rules ເຊັ່ນວ່າ ການແກ້ໄຂຕາຕະລາງເງິນເດືອນພະນັກງານຈະຕ້ອງແຈ້ງອີເມວໄປຫາຫົວໜ້າຝ່າຍການເງິນ

Triggers

◆ DML Trigger

■ ການສ້າງ DML Trigger ມີຮູບແບບດັ່ງນີ້

```
CREATE TRIGGER [schema_name.] trigger_name
ON {table | view}
[WITH <dml_trigger_option> [...n]]
{FOR | AFTER | INSTEAD OF}
{[INSERT] [ , ] [UPDATE] [ , ] [DELETE]}
[WITH APPEND]
[NOT FOR REPLICATION]
AS {sql_statement[:][,...n]}EXTERNAL NAME <method specifier>[:]}
<ddl_trigger_option>::=
    [ENCRYPTION]
    [EXECUTE AS Clause]
<method_option>::=
    assembly_name.class_name.method_name
```

Triggers

◆ DML Trigger

■ ການສ້າງ DML Trigger

- ◆ ຕົວຢ່າງ ການໃຊ້ຄໍາສັ່ງ Transact-SQL ສ້າງ DML Trigger ທີ່ສົ່ງອີເມວໃຫ້ Mala ທີ່ເປັນຫົວໜ້າການເງິນ ເມື່ອຕາຕະລາງເງິນເດືອນພະນັກງານ (Employee_Salary) ມີການປ່ຽນແປງ

```
CREATE TRIGGER reminder_Employee_Salary  
ON HR.Employee_Salary  
AFTER INSERT, UPDATE, DELETE  
AS  
EXEC master..xp_sendmail 'Mala',  
    'Employee_Salary Table was modified.'
```

Triggers

◆ DML Trigger

■ ການສ້າງ DML Trigger

- ◆ ຕົວຢ່າງ ການໃຊ້ຄໍາສັ່ງ Transact-SQL ສ້າງ DML Trigger ທີ່ໃຊ້ກວດສອບຕາຕະລາງພະນັກງານໃນຖານຂໍ້ມູນຊື່ huresource ເມື່ອມີການເພີ່ມ, ລຶບ ແລະ ປັບປຸງຂໍ້ມູນຂໍ້ມູນພະນັກງານ
- ◆ ທໍາອິດເຮົາສ້າງຕາຕະລາງດັ່ງນີ້

```
CREATE TABLE Employees (  
    EmployeeID integer NOT NULL IDENTITY(1, 1) ,  
    EmployeeName VARCHAR(50) ,  
    EmployeeAddress VARCHAR(50) ,  
    MonthSalary NUMERIC(10, 2)  
    PRIMARY KEY CLUSTERED (EmployeeID)  
)  
GO
```

Triggers

◆ DML Trigger

■ ການສ້າງ DML Trigger

- ◆ ສ້າງຕາຕະລາງ EmployeesAudit ເພື່ອບັນທຶກການປ່ຽນແປງທີ່ເກີດຂຶ້ນ ດັ່ງນີ້

```
CREATE TABLE EmployeesAudit (  
    AuditID INTEGER NOT NULL IDENTITY(1, 1) ,  
    EmployeeID INTEGER ,  
    EmployeeName VARCHAR(50) ,  
    EmployeeAddress VARCHAR(50) ,  
    MonthSalary NUMERIC(10, 2) ,  
    ModifiedBy VARCHAR(128) ,  
    ModifiedDate DATETIME ,  
    Operation CHAR(1)  
    PRIMARY KEY CLUSTERED ( AuditID )  
)  
GO
```


Triggers

◆ DML Trigger

■ ການສ້າງ DML Trigger

- ◆ ເພີ່ມຂໍ້ມູນເຂົ້າໃນຕາຕະລາງ Employees ເພື່ອທົດສອບ ດັ່ງນີ້

```
INSERT INTO dbo.Employees
```

```
    ( EmployeeName ,  
      EmployeeAddress ,  
      MonthSalary
```

```
)
```

```
SELECT 'Khamdy Duangvilay', 'Dongdok, Xaythany, Vientiane Capital', 1100000
```

```
UNION ALL
```

```
SELECT 'Mala Vilaphanh', 'Symungkhune, Parksan, Bolikhamxay', 1300000
```

```
UNION ALL
```

```
SELECT 'Kinnaly Syvilay', 'Tanmyxay, Xaythany, Vientiane Capital', 1500000
```

```
UNION ALL
```

```
SELECT 'Kanha Sysongkham', 'Chommany, Saysettha, Vientiane Capital', 1200000
```

```
GO
```

Triggers

◆ DML Trigger

SQLQuery2.sql - Wl...uresource (sa (55))* x

```
use huresource
go
INSERT INTO dbo.Employees
    ( EmployeeName ,
      EmployeeAddress ,
      MonthSalary
    )
SELECT 'Khamdy Duangvilay', 'Dongdok, Xaythany, Vientiane Capital', 1100000
UNION ALL
SELECT 'Mala Vilaphanh', 'Symungkhune, Parksan, Bolikhamxay', 1300000
UNION ALL
SELECT 'Kinnaly Syvilay', 'Tanmyxay, Xaythany, Vientiane Capital', 1500000
UNION ALL
SELECT 'Kanha Sysongkham', 'Chommany, Saysettha, Vientiane Capital', 1200000
GO
```

100 % <

Messages

(4 row(s) affected)

Triggers

DML Trigger

- ການສ້າງ DML Trigger
 - ◆ ເພີ່ມຂໍ້ມູນເຂົ້າໃນຕາຕະລາງ Employees ເພື່ອທົດສອບ ດັ່ງນີ້

	EmployeeID	EmployeeName	EmployeeAddress	MonthSalary
▶	1	Khamdy Duangvilay	Dongdok, Xaythany, Vientiane Capital	1100000.00
	2	Mala Vilaphanh	Symungkhune, Parksan, Bolikhamxay	1300000.00
	3	Kinnaly Syvilay	Tanmyxay, Xaythany, Vientiane Capital	1500000.00
	4	Kanha Sysongkham	Chommany, Saysettha, Vientiane Capital	1200000.00
*	NULL	NULL	NULL	NULL

[illegible]

Triggers

◆ DML Trigger

■ ການສ້າງ DML Trigger

◆ ສ້າງ DML Trigger ເພື່ອກວດສອບ ດັ່ງນີ້

```
CREATE TRIGGER TR_Audit_Employees ON dbo.Employees  
FOR INSERT, UPDATE, DELETE  
AS
```

```
DECLARE @login_name VARCHAR(128)
```

```
SELECT @login_name = login_name  
FROM sys.dm_exec_sessions  
WHERE session_id = @@SPID
```

Triggers

◆ DML Trigger

■ ການສ້າງ DML Trigger

◆ ສ້າງ DML Trigger ເພື່ອກວດສອບ ດັ່ງນີ້

```
IF EXISTS ( SELECT 0 FROM Deleted )
BEGIN
    IF EXISTS ( SELECT 0 FROM Inserted )
    BEGIN
        INSERT INTO dbo.EmployeesAudit
        ( EmployeeID ,
          EmployeeName ,
          EmployeeAddress ,
          MonthSalary ,
          ModifiedBy ,
          ModifiedDate ,
          Operation
        )
```

Triggers

◆ DML Trigger

- ການສ້າງ DML Trigger
 - ◆ ສ້າງ DML Trigger ເພື່ອກວດສອບ ດັ່ງນີ້

```
SELECT D.EmployeeID ,  
        D.EmployeeName ,  
        D.EmployeeAddress ,  
        D.MonthSalary ,  
        @login_name ,  
        GETDATE() ,  
        'U'  
FROM Deleted D  
END
```

Triggers

◆ DML Trigger

- ການສ້າງ DML Trigger
 - ◆ ສ້າງ DML Trigger ເພື່ອກວດສອບ ດັ່ງນີ້

ELSE

BEGIN

INSERT INTO dbo.EmployeesAudit

(EmployeeID ,
EmployeeName ,
EmployeeAddress ,
MonthSalary ,
ModifiedBy ,
ModifiedDate ,
Operation
)

Triggers

◆ DML Trigger

- ການສ້າງ DML Trigger
 - ◆ ສ້າງ DML Trigger ເພື່ອກວດສອບ ດັ່ງນີ້

```
SELECT D.EmployeeID ,  
        D.EmployeeName ,  
        D.EmployeeAddress ,  
        D.MonthSalary ,  
        @login_name ,  
        GETDATE() ,  
        'D'  
FROM Deleted D  
END  
END
```


Triggers

◆ DML Trigger

- ການສ້າງ DML Trigger
 - ◆ ສ້າງ DML Trigger ເພື່ອກວດສອບ ດັ່ງນີ້

```
ELSE
    BEGIN
        INSERT INTO dbo.EmployeesAudit
            ( EmployeeID ,
              EmployeeName ,
              EmployeeAddress ,
              MonthSalary ,
              ModifiedBy ,
              ModifiedDate ,
              Operation
            )
```

Triggers

◆ DML Trigger

- ການສ້າງ DML Trigger
 - ◆ ສ້າງ DML Trigger ເພື່ອກວດສອບ ດັ່ງນີ້

```
SELECT I.EmployeeID ,  
       I.EmployeeName ,  
       I.EmployeeAddress ,  
       I.MonthSalary ,  
       @login_name ,  
       GETDATE() ,  
       'I'  
FROM   Inserted I  
  
END  
  
GO
```

Triggers

◆ DML Trigger

■ ການສ້າງ DML Trigger

```
WINSQL.huresourc...bo.EmployeesAudit  WINSQL.huresource - dbo.Employees  SQLQuery2.sql - WI...uresource (sa (55))* X

use huresource
go
CREATE TRIGGER TR_Audit_Employees ON dbo.Employees
FOR INSERT, UPDATE, DELETE
AS
DECLARE @login_name VARCHAR(128)

SELECT @login_name = login_name
FROM sys.dm_exec_sessions
WHERE session_id = @@SPID

IF EXISTS ( SELECT 0 FROM Deleted )
BEGIN
    IF EXISTS ( SELECT 0 FROM Inserted )
    BEGIN
        INSERT INTO dbo.EmployeesAudit
            ( EmployeeID ,
              EmployeeName ,
              EmployeeAddress ,
              MonthSalary ,
              ModifiedBy ,
              ModifiedDate ,
              Operation
            )
    
```

100 % <

Messages
Command(s) completed successfully.

Triggers

◆ DML Trigger

■ ການສ້າງ DML Trigger

- ◆ ທົດລອງເພີ່ມຂໍ້ມູນເຂົ້າໃນຕາຕະລາງ Employees ດັ່ງນີ້

```
INSERT INTO dbo.Employees
```

```
    ( EmployeeName ,  
      EmployeeAddress ,  
      MonthSalary
```

```
)
```

```
SELECT 'Vidany Singhakhom', 'Donenoun, Xaythany, Vientiane Capital', 1400000
```

```
UNION ALL
```

```
SELECT 'Touly Vang', 'Naluang, Kham, Xiengkhoung', 1300000
```

```
UNION ALL
```

```
SELECT 'Nengvang Toulao', 'Tanmyxay, Xaythany, Vientiane Capital', 1200000
```

```
UNION ALL
```

```
SELECT 'Viengkham Sysongkham', 'Chommany, Saysettha, Vientiane Capital', 1200000
```

```
GO
```

Triggers

◆ DML Trigger

```
use huresource
go
INSERT INTO dbo.Employees
( EmployeeName ,
  EmployeeAddress ,
  MonthSalary
)
SELECT 'Vidany Singhakhom', 'Donenoun, Xaythany, Vientiane Capital', 1400000
UNION ALL
SELECT 'Touly Vang', 'Naluang, Kham, Xiengkhoung', 1300000
UNION ALL
SELECT 'Nengvang Toulao', 'Tanmyxay, Xaythany, Vientiane Capital', 1200000
UNION ALL
SELECT 'Viengkham Sysongkham', 'Chommany, Saysettha, Vientiane Capital', 1200000
GO
```

100 % <

Messages

(4 row(s) affected)

(4 row(s) affected)

Triggers

◆ DML Trigger

■ ການສ້າງ DML Trigger

- ◆ ທົດລອງເພີ່ມຂໍ້ມູນເຂົ້າໃນຕາຕະລາງ Employees ດັ່ງນີ້

WINSQL.huresource - dbo.Employees X

SQLQuery2.sql - Wl...uresource (sa (55))*

	EmployeeID	EmployeeName	EmployeeAddress	MonthSalary
▶	1	Khamdy Duangvilay	Dongdok, Xaythany, Vientiane Capital	1100000.00
	2	Mala Vilaphanh	Symungkhune, Parksan, Bolikhamxay	1300000.00
	3	Kinnaly Syvilay	Tanmyxay, Xaythany, Vientiane Capital	1500000.00
	4	Kanha Sysongkham	Chommany, Saysettha, Vientiane Capital	1200000.00
	5	Vidany Singhakhom	Donenoun, Xaythany, Vientiane Capital	1400000.00
	6	Touly Vang	Naluang, Kham, Xiengkhoung	1300000.00
	7	Nengvang Toulao	Tanmyxay, Xaythany, Vientiane Capital	1200000.00
	8	Viengkham Sysongkham	Chommany, Saysettha, Vientiane Capital	1200000.00
*	NULL	NULL	NULL	NULL

Triggers

◆ DML Trigger

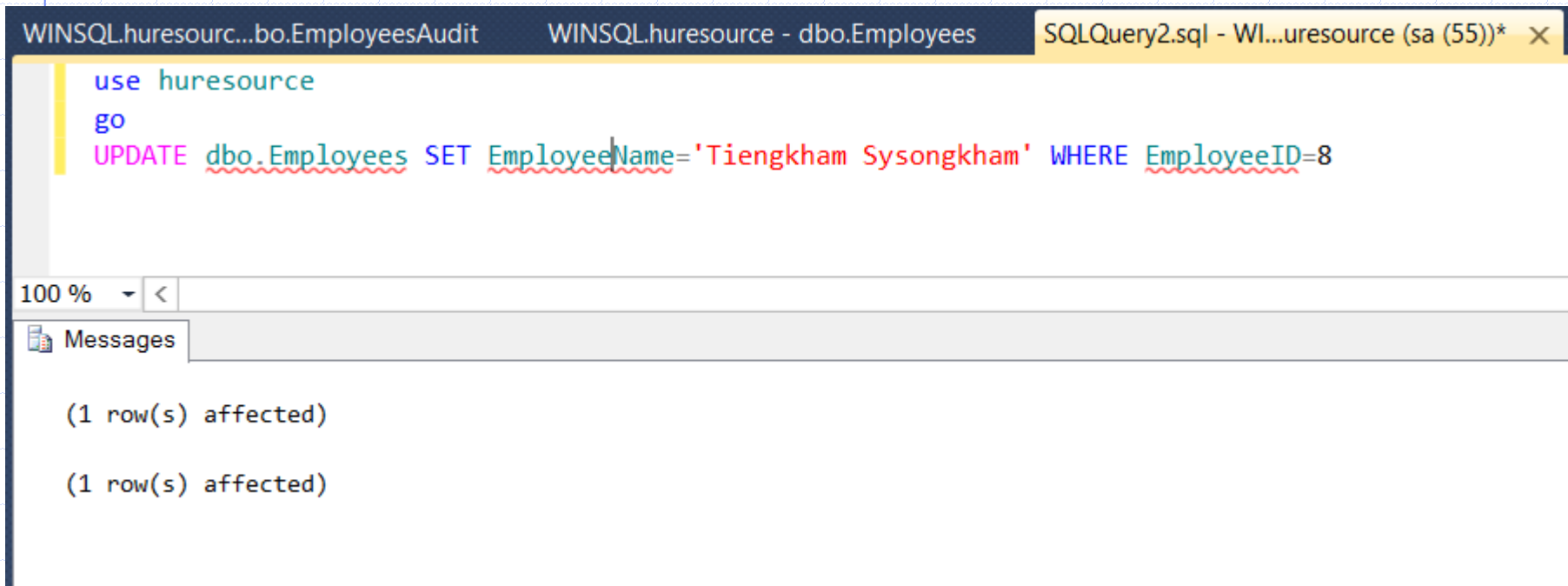
- ການສ້າງ DML Trigger
 - ◆ ທົດລອງເພີ່ມຂໍ້ມູນເຂົ້າໃນຕາຕະລາງ Employees ດັ່ງນີ້

WINSQL.huresourc...bo.EmployeesAudit X WINSQL.huresource - dbo.Employees SQLQuery2.sql - WI...uresource (sa (55))*								
	AuditID	EmployeeID	EmployeeName	EmployeeAddress	MonthSalary	ModifiedBy	ModifiedDate	Operation
▶	1	8	Viengkham Sysongkham	Chommany, Saysettha,...	1200000.00	sa	2020-11-06 03:...	I
	2	7	Nengvang Toulao	Tanmyxay, Xaythany, V...	1200000.00	sa	2020-11-06 03:...	I
	3	6	Touly Vang	Naluang, Kham, Xieng...	1300000.00	sa	2020-11-06 03:...	I
	4	5	Vidany Singhakhom	Donenoun, Xaythany, ...	1400000.00	sa	2020-11-06 03:...	I
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Triggers

◆ DML Trigger

- ການສ້າງ DML Trigger
 - ◆ ທົດລອງອັບເດດຂໍ້ມູນໃນຕາຕະລາງ Employees ດັ່ງນີ້



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows the 'WINSQL.huresource...bo.EmployeesAudit' and 'WINSQL.huresource - dbo.Employees' folders. The bottom pane shows the 'SQLQuery2.sql - WI...uresource (sa (55))*' window with the following SQL query:

```
use huresource
go
UPDATE dbo.Employees SET EmployeeName='Tiengkham Sysongkham' WHERE EmployeeID=8
```

Below the query window, the 'Messages' pane shows the execution results:

```
(1 row(s) affected)
(1 row(s) affected)
```


Triggers



DML Trigger

- ການສ້າງ DML Trigger

WINSQL.huresource - dbo.Employees ✕

SQLQuery2.sql - Wl...uresource (sa (55))*

	EmployeeID	EmployeeN...	EmployeeAddress	MonthSalary
▶	1	Khamdy Du...	Dongdok, Xaythany, Vientiane Capital	1100000.00
	2	Mala Vilaph...	Symungkhune, Parksan, Bolikhamxay	1300000.00
	3	Kinnaly Syvi...	Tanmyxay, Xaythany, Vientiane Capital	1500000.00
	4	Kanha Syso...	Chommany, Saysettha, Vientiane Capital	1200000.00
	5	Vidany Sing...	Donenoun, Xaythany, Vientiane Capital	1400000.00
	6	Touly Vang	Naluang, Kham, Xiengkhoung	1300000.00
	7	Nengvang T...	Tanmyxay, Xaythany, Vientiane Capital	1200000.00
	8	Tiengkham ...	Chommany, Saysettha, Vientiane Capital	1200000.00
*	NULL	NULL	NULL	NULL

Triggers

◆ DML Trigger

■ ການສ້າງ DML Trigger

- ◆ ທົດລອງອັບເດດຂໍ້ມູນໃນຕາຕະລາງ Employees ດັ່ງນີ້

WINSQL.huresourc...bo.EmployeesAudit X WINSQL.huresource - dbo.Employees SQLQuery2.sql - WI...uresource (sa (55)								
	AuditID	EmployeeID	EmployeeN...	EmployeeA...	MonthSalary	ModifiedBy	ModifiedD...	Operation
▶	1	8	Viengkham ...	Chommany,...	1200000.00	sa	2020-11-06 ...	I
	2	7	Nengvang T...	Tanmyxay, ...	1200000.00	sa	2020-11-06 ...	I
	3	6	Touly Vang	Naluang, Kh...	1300000.00	sa	2020-11-06 ...	I
	4	5	Vidany Sing...	Donenoun, ...	1400000.00	sa	2020-11-06 ...	I
	5	8	Viengkham ...	Chommany,...	1200000.00	sa	2020-11-06 ...	U
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Triggers

◆ DML Trigger

- ການແກ້ໄຂ DML Trigger ມີຮູບແບບຄໍາສັ່ງດັ່ງນີ້

```
ALTER TRIGGER [schema_name.] trigger_name
ON {table | view}
[WITH <dml_trigger_option> [...n]]
{FOR | AFTER | INSTEAD OF}
{[INSERT] [ , ] [UPDATE] [ , ] [DELETE]}
[WITH APPEND]
[NOT FOR REPLICATION]
AS {sql_statement[:][,...n]}EXTERNAL NAME <method specifier>[:]}
<ddl_trigger_option>::=
    [ENCRYPTION]
    [EXECUTE AS Clause]
<method_option>::=
    assembly_name.class_name.method_name
```

Triggers

◆ DML Trigger

- ການແກ້ໄຂ DML Trigger
 - ◆ ຕົວຢ່າງ ການແກ້ໄຂ DML Trigger ດ້ວຍຄໍາສັ່ງ Transact

```
ALTER TRIGGER Sales.bonus_reminder  
ON Sales.SalesPersonQuotaHistory  
AFTER INSERT  
AS RAISERROR ('Notify Compensation', 16, 10);
```

Triggers

◆ DML Trigger

■ ການລຶບ DML Trigger

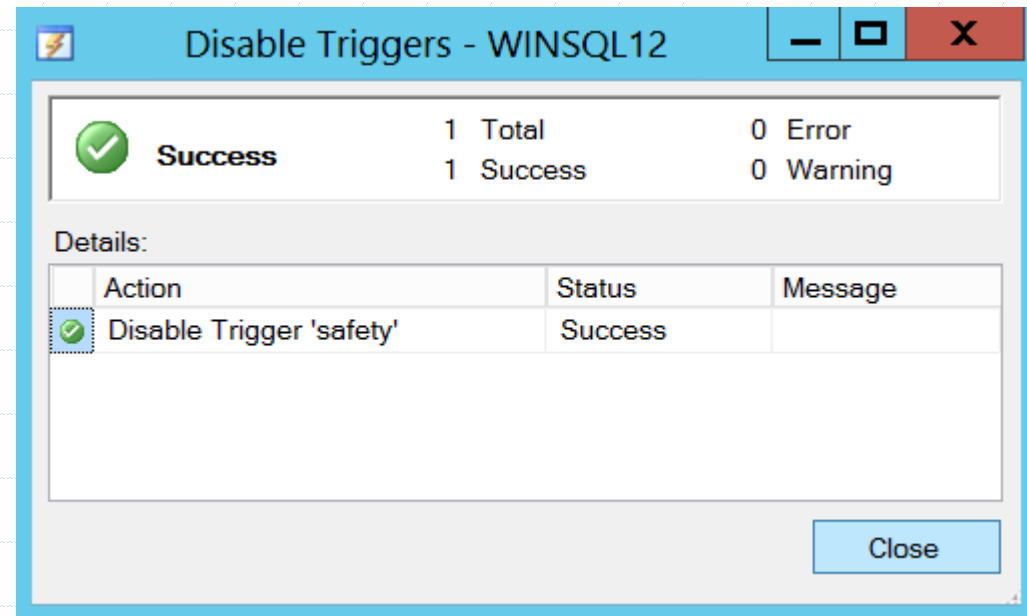
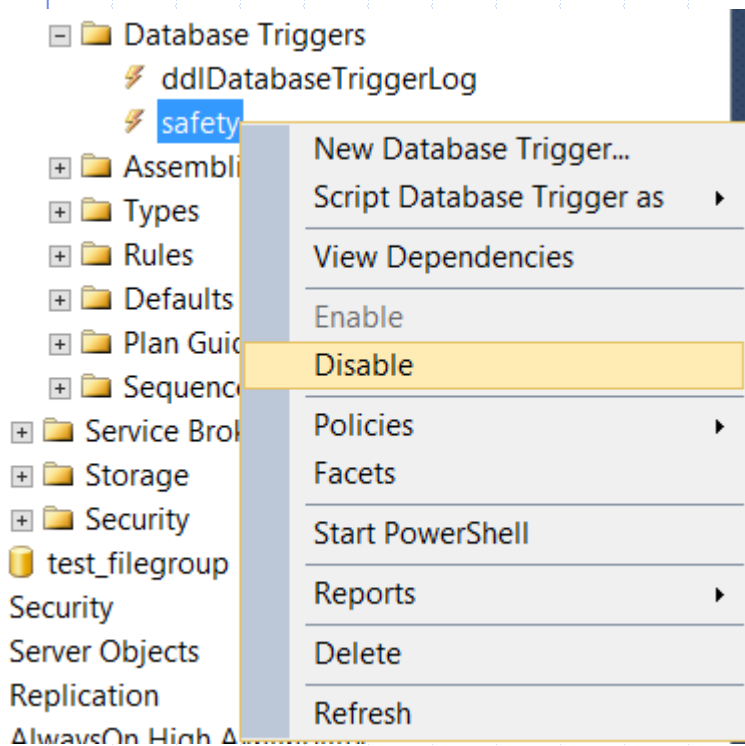
1. ເຂົ້າໄປໃນຕາຕະລາງທີ່ມີ DML Trigger ທີ່ຕ້ອງການລຶບ
2. ເຂົ້າໄປໃນໂຟນເດີ Triggers
3. ໃຫ້ຄົກຂວາໃສ່ DML Trigger ທີ່ຕ້ອງການລຶບ
4. ຄົກ OK ເພື່ອຢືນຢັນການລຶບ

Triggers

- ◆ ການ Enable-Disable Trigger
 - ການ Enable-Disable ໂດຍໃຊ້ SQL Management Studio ນັ້ນ ສາມາດເຮັດໄດ້ສະເພາະ DML Trigger ໂດຍການ ຄິກຂວາໃສ່ DML Trigger ທີ່ຕ້ອງການ ແລ້ວເລື່ອຄໍາສັ່ງ Enable/Disable

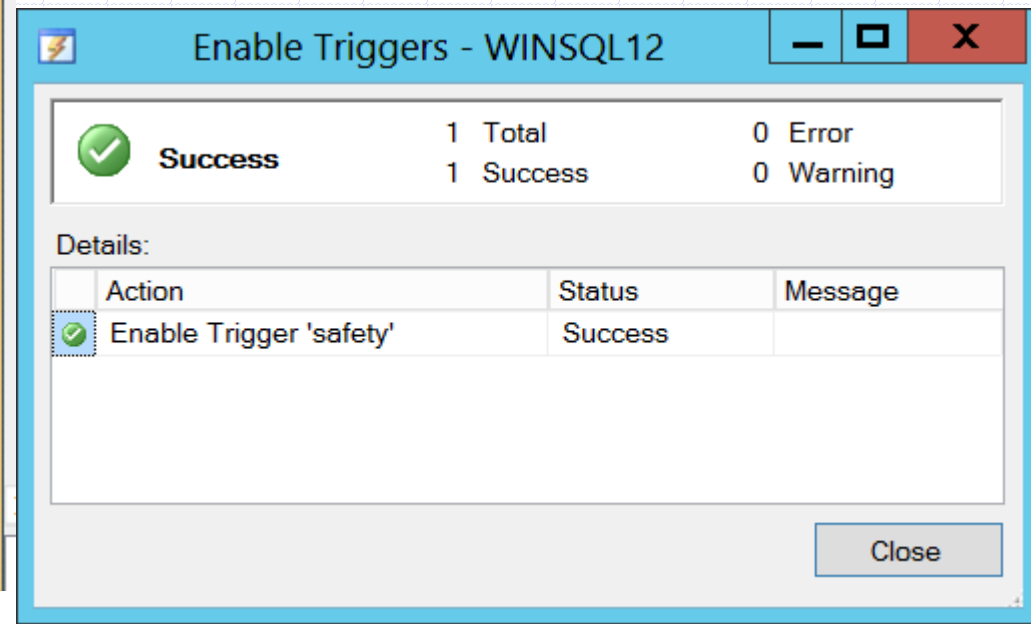
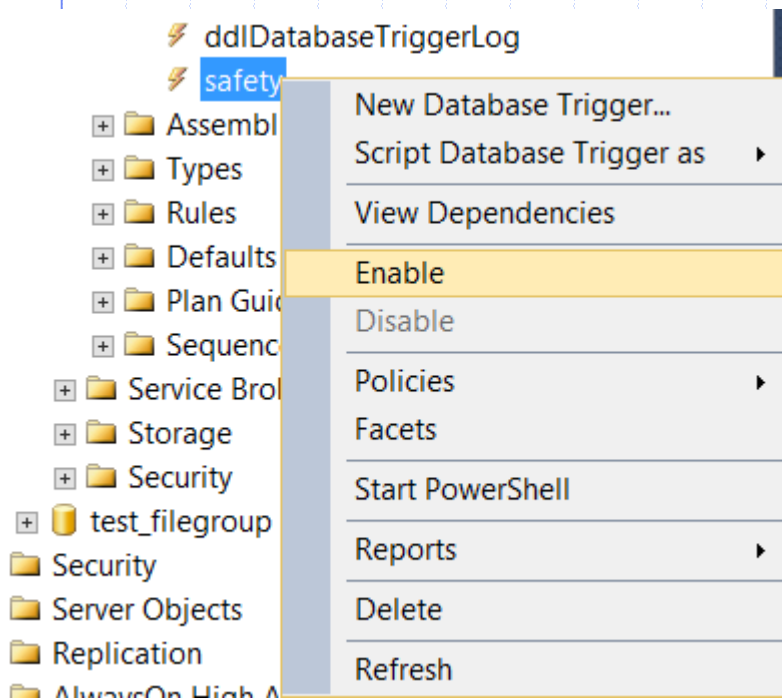
Triggers

ການ Enable-Disable Trigger



Triggers

◆ ການ Enable-Disable Trigger



Triggers

◆ ການ Enable-Disable Trigger

- ການ Enable ໂດຍໃຊ້ Transact-SQL ມີຮູບແບບດັ່ງນີ້
ENABLE TRIGGER {[schema_name.] trigger_name [,...n] | ALL}
ON {object_name | DATABASE | ALL SERVER} [;]
- ຕົວຢ່າງ ການ Enable DML Trigger ໃນຕາຕະລາງຊື່
Person.Address ໃນຖານຂໍ້ມູນ AdventureWorks2012
USE AdventureWorks2012
ENABLE TRIGGER Person.Address ON DATABASE
- ຕົວຢ່າງ ການ Enable DDL Trigger ຊື່ safety ໃນຖານຂໍ້ມູນ
AdventureWorks2012
USE AdventureWorks2012
ENABLE TRIGGER safety ON DATABASE

Triggers

◆ ການ Enable-Disable Trigger

- ການ Disable ໂດຍໃຊ້ Transact-SQL ມີຮູບແບບດັ່ງນີ້
DISABLE TRIGGER {[schema_name.] trigger_name [,...n] | ALL}
ON {object_name | DATABASE | ALL SERVER} [;]
- ຕົວຢ່າງ ການ Disable DML Trigger ໃນຕາຕະລາງຊື່
Person.Address ໃນຖານຂໍ້ມູນ AdventureWorks2012
USE AdventureWorks2012
DISABLE TRIGGER Person.Address ON DATABASE
- ຕົວຢ່າງ ການ Disable DDL Trigger ຊື່ safety ໃນຖານຂໍ້ມູນ
AdventureWorks2012
USE AdventureWorks2012
DISABLE TRIGGER safety ON DATABASE

Triggers

◆ CLR Trigger

- ເປັນ DML Trigger/DDL Trigger ທີ່ຄໍາສັ່ງໃນ Trigger ເປັນການໃຊ້ງານ Method ຂອງ Assemblies ໃນ .NET Framework Common Language Runtime (CLR)
- ໃນການເຮັດວຽກກັບ CLR ຂອງ SQL Server 2012 ນັ້ນ ຈະຕ້ອງໄດ້ໄປ Enable CLR ເສຍກ່ອນດັ່ງນີ້
USE master
EXEC sp_configure 'clr enabled', 1
Reconfigure

Triggers

◆ CLR Trigger

- ການສ້າງ CLR Trigger (DML ຫຼື DDL) ມີຂັ້ນຕອນດັ່ງນີ້
 1. ສ້າງ Trigger ໂດຍໃຊ້ພາສາທີ່ສະໜັບສະໜູນຕາມຄລາດຂອງ .NET Framework ແລ້ວຄອມພາຍຄລາດເຫຼົ່ານັ້ນເປັນຟາຍ .dll
 2. ເອົາຟາຍ .dll ມາ Register ໃນ SQL Server ໂດຍໃຊ້ຄໍາສັ່ງ CREATE ASSEMBLY
 3. ສ້າງ Trigger ມາເອິ້ນໃຊ້ງານ Method CLR ຂອງເຮົາໄປເຮັດວຽກ ໂດຍໃຊ້ CREATE TRIGGER

Triggers

◆ CLR Trigger

- ຕົວຢ່າງ ການສ້າງ CLR Trigger ທີ່ເຮັດວຽກເມື່ອຕາຕະລາງ Person.Contact ຖືກແກ້ໄຂ

- ◆ Visual Basic .NET

```
<SqlTrigger(Name:=“ContactUpdTrg”, Target:=“Person.Contact”, Event:=“FOR UPDATE”)> _  
Public Share Sub ChangeEmail()  
Dim trg AS SqlTriggerContext = SqlContext.GetTriggerContext()  
If (trg.TriggerAction=TriggerAction.Update) then  
    Dim SqlComm As SqlCommand = SqlContext.GetCommand()  
    SqlComm.CommandText = “SELECT emailAddress FROM INSERTED”  
    Dim rdr As SqlDataReader = SqlComm.ExecuteReader()  
    While (rdr.Read())  
        ‘send e-mail to each updated contact  
    End while  
End if  
End Sub
```

Triggers

◆ CLR Trigger

- ຕົວຢ່າງ ການສ້າງ CLR Trigger ທີ່ເຮັດວຽກເມື່ອຕາຕະລາງ Person.Contact ຖືກແກ້ໄຂ

- ◆ Visual C#

```
[SqlTrigger(Name:="ContactUpdTrg", Target:="Person.Contact", Event:="FOR UPDATE")]
Public static void ChangeEmail() {
    SqlTriggerContext trg = SqlContext.GetTriggerContext();
    If (trg.TriggerAction==TriggerAction.Update) {
        SqlCommand SqlComm = SqlContext.GetCommand();
        SqlComm.CommandText = "SELECT emailAddress FROM INSERTED";
        SqlDataReader rdr = SqlComm.ExecuteReader();
        While (rdr.Read()){
            //send e-mail to each updated contact
        }
    }
}
```

Triggers

◆ CLR Trigger

- ຕົວຢ່າງ ການສ້າງ CLR Trigger ທີ່ເຮັດວຽກເມື່ອຕາຕະລາງ Person.Contact ຖືກແກ້ໄຂ
 - ◆ ຄໍາສັ່ງຂ້າງເທິງເປັນການກຳນົດການເຮັດວຽກຂອງ Trigger ຊຶ່ງຈະຕ້ອງໄດ້ເລືອກ Build Solution ເພື່ອ Compile ໃຫ້ເປັນຟາຍ .dll ແລ້ວເອົາມາ Import Assembly ກັບຖານຂໍ້ມູນ ຈຶ່ງຈະເຮັດໃຫ້ຖານຂໍ້ມູນຮູ້ຈັກ Method CLR ຂອງເຮົາ
 - ◆ ແລ້ວເອົາມາ Link ເຂົ້າກັບ Trigger ໃນຖານຂໍ້ມູນດ້ວຍຄໍາສັ່ງ Transact-SQL ເພື່ອໃຫ້ Trigger ມາເອີ້ນໃຊ້ງານ Method CLR ຂອງເຮົາໄປເຮັດວຽກ

Triggers

◆ CLR Trigger

- ຕົວຢ່າງ ການສ້າງ CLR Trigger ທີ່ເຮັດວຽກເມື່ອຕາຕະລາງ Person.Contact ຖືກແກ້ໄຂ

- ◆ ຕົວຢ່າງ

```
CREATE TRIGGER Person.ContactUpdTrg ON  
    Person.Contact  
FOR  
EXTERNAL NAME  
    Contacts.[PersonData.Triggers].ChangeEmail
```


Constraints

- ◆ ເປັນອອບເຈັກໜຶ່ງທີ່ໃຊ້ໃນການຄວບຄຸມຄວາມຖືກຕ້ອງຂອງຂໍ້ມູນ
- ◆ ສາມາດກຳນົດ Constraints ໃນຄໍລໍາຕອນທີ່ສ້າງຕາຕະລາງ ຫຼື ກຳນົດໃນຕາຕະລາງທີ່ມີຢູ່ແລ້ວກໍໄດ້
- ◆ Constraints ທີ່ເຮົາສາມາດກຳນົດໄດ້ປະກອບດ້ວຍ
 - Primary Key Constraints
 - Foreign Key Constraints
 - Unique Constraints
 - Check Constraints
 - Default Definitions
 - Allowing Null Values

Constraints

◆ Primary Key Constraints

- ເປັນ Key ທີ່ເກີດຈາກ 1 ຫຼື ຫຼາຍຄໍລ່າ ທີ່ໃຊ້ໃນການອ້າງອີງເຖິງຂໍ້ມູນໃນແຕ່ລະແຖວໃນຕາຕະລາງ ເຮັດໃຫ້ແຕ່ລະແຖວມີຄ່າບໍ່ຊ້ຳກັນ
- ຈະກຳນົດໃນຂະນະສ້າງຕາຕະລາງຫຼືຫຼັງຈາກສ້າງຕາຕະລາງກໍໄດ້
- ໃນຕາຕະລາງໜຶ່ງ ມີພຽງ 1 Primary Key ເທົ່ານັ້ນ

◆ Foreign Key Constraints

- ເປັນ 1 ຫຼື ຫຼາຍຄໍລ່າຮ່ວມກັນທີ່ໃຊ້ໃນການເຊື່ອມຕໍ່ຄວາມສຳພັນຂອງສອງຕາຕະລາງ
- ນອກຈາກຈະກຳນົດຈາກ Primary Key ຂອງຕາຕະລາງຫຼັກແລ້ວ ຍັງສາມາດກຳນົດຈາກຄໍລ່າທີ່ກຳນົດເປັນ Unique Constraints ນຳອີກ

Constraints

◆ Unique Constraints

- ນອກຈາກ Primary Key Constraints ແລ້ວ ເຮົາສາມາດກຳນົດ Unique Constraints ເພື່ອໃຫ້ຂໍ້ມູນໃນຄໍລໍ່ານັ້ນມີຄ່າບໍ່ຊ້ຳກັນໄດ້
- ສາມາດກຳນົດໃນ 1 ຫຼື ຫຼາຍຄໍລໍ່າ ແລະ ສາມາດກຳນົດໃຫ້ເປັນ Null ໄດ້

◆ Check Constraints

- ເປັນ Domain Integrity ທີ່ກຳນົດຄ່າທີ່ຍອມຮັບໄດ້ໃນຄໍລໍ່າ
- ຄ້າຍຄືກັບ Foreign Key ຄ່າໃນຕາຕະລາງສໍາຮອງຈະຕ້ອງມີຢູ່ໃນຕາຕະລາງຫຼັກ
- ແຕກຕ່າງຈາກ Foreign Key ຢູ່ບ່ອນວ່າ Check Constraints ເປັນການກຳນົດກົດຂັ້ນມາເພື່ອບັງຄັບຂໍ້ມູນໃນຄໍລໍ່ານັ້ນ

Constraints

◆ Default Definitions

- ເໝືອນກັບການກຳນົດ Default values ໃຫ້ກັບຄໍລ່າ
- ໃນກໍລະນີທີ່ມີການປ້ອນຄ່າ Null ໃນຄໍລ່າທີ່ມີການກຳນົດຄ່າ Default ໄວ້ ມັນຈະເອົາຄ່າ Default ໄປເກັບແທນ
- ມັກຖືກເອົາໄປໃຊ້ໃນຄໍລ່າທີ່ມີການກຳນົດບໍ່ຍອມຮັບຄ່າ Null

◆ Allowing Null Values

- ເປັນການກຳນົດໃນລະດັບຄໍລ່າວ່າ ຄໍລ່ານັ້ນອານຸຍາດໃຫ້ເປັນຄ່າ Null ຫຼື ບໍ່
- ກຳນົດໂດຍໃຊ້ Keyword
 - ◆ NULL ຄໍລ່ານັ້ນສາມາດເປັນຄ່າ NULL ໄດ້
 - ◆ NOT NULL ຄໍລ່ານັ້ນບໍ່ສາມາດເປັນຄ່າ NULL ໄດ້